

Week 3 - Suggested Homework

Review of Python Fundamentals and Intro to Pandas

Review of **Python Fundamentals** (~1-2 hours)

Depending on your progress so far, review "Intro to Python" videos recommended earlier. This week, focus on user-defined functions and third-party packages (e.g., numpy, re, math, time, etc.). These videos and exercises from Week 2 correspond to the difficulty of the quiz and more generally, the conceptual understanding of Python fundamentals expected in first month. The main topics are **objects** in base Python, **conditional statements**, **loops**, and **functions**. It also implies that you understand how to write code in Python language more generally (e.g., indentation, case sensitivity, assigning objects to variables, # for comments, etc.).

- *Learn Coding with Python in 1 Hour* (@programmingwithmosh): [link](#)
- *Python Full Course for Beginners [2025]* (@programmingwithmosh) : [link](#)

You are **not** expected to memorize any specific functions or methods, but you should develop an intuition how to read simple Python code and imagine what it does. I will not ask tricky questions to test your memory of the exact syntax or edge cases, but for example, you should have a "feeling" that `my_string.upper()` returns `my_string` in upper case or that a *for loop* can iterate through different object types (e.g., range, list, string, etc.) or that functions might modify object(s) in place or return a new object.

Reading **Tidy Data** ([PDF link](#)) by Hadley Wickham (~1 hour)

Hadley Wickham is the creator of RStudio and *tidyverse* (dplyr, ggplot, tidyr, etc.). His "Tidy Data" paper is the key reading that every *good* data analyst should start with. I was told to read it on my first day of my first internship, and I assumed that everyone does that. Ever since, I realized that few analysts are familiar with this basic philosophy of "tidy data", and those few analysts are always among the best ones at their job.

The main goal of this paper is to standardize what tidy data should look like (p. 4):

1. *Each variable forms a column.*
2. *Each observation forms a row.*
3. *Each type of observational unit forms a table.*

It emphasizes the idea that each tidy dataset should look exactly the same, if cleaned properly. It also rebutes the idea that "data cleaning is hard because each messy dataset is so unique". In reality, analysts often feel that data cleaning is hard, because they do not have a clear end goal in mind, and think that for each analysis they need a *different* "clean" dataset. This paper resolves this exact issue, by providing a *standard* framework of what all clean data should look like. You then start realizing that all messy datasets share a few common patterns, and you can always apply the same 3-5 steps to get from any messy dataset to a clean one. Your main job is to recognize these patterns, and efficiently apply the same few data cleaning tools. Then, starting from a tidy dataset, any data analysis can be usually done in a couple of lines of code.

All of *tidyverse* in R and *pandas* in Python are built around this philosophy, therefore, understanding the intuition behind these software tools makes you much more efficient in using them. The paper also gives a few examples of most common messy-to-tidy data cleaning patterns, e.g., values in variable names, multiple variables in a single column, etc.

Note that this paper uses **functions from R**, so you do not need to memorize them, but *pandas* has nearly identical functions, just under different names. While reading, you should easily see a distinction of universal concepts (e.g., variables, observations, melting, reshaping, etc.) vs. exact functions that implement these concepts (e.g., *transform()*, *summarise()*, etc.). The former applies to all languages, while the latter varies by language. You can ignore the code parts in 5. *Case study*, since it is a bit outdated (preferred style of using *tidyverse* functions in R have changed a bit).

Intro to Pandas (~2-3 hours)

Theory

Below are the main official resources for learning *pandas* - all from <https://pandas.pydata.org/>. I would recommend starting with the *10 min tutorial* and all of *Getting started tutorials* (they are very concise!). Then once you have a general sense of how *pandas* work and what it can do, you should practice applying it on a dataset of your choice. To guide your work with data, you can pick a specific topic from the *User Guide* and practice concepts from one topic at a time (e.g., next week we will focus on *reshaping*). Other resources are for reference.

- [10min tutorial for a quick start](#)
- [Getting started tutorials](#)
- [User Guide](#)
- [Pandas Cheat Sheet](#) (note "Tidy Data" at the very top - now you know what it means!)
- [API reference](#)
- [Community tutorials](#)
- [Comparison with other tools](#)

If you prefer watching videos, below are a couple of comprehensive playlists:

- [Pandas | Data Analysis in Python](#) (from @coreyms)
- [Pandas Tutorial \(Data Analysis In Python\)](#) (from @codebasics)

Practice

To get into the habit of planning your data analysis projects efficiently, I would recommend following the steps based on a real-world scenario. After you review these sample steps, start writing a text document where you define an analysis of your choice, and then create a Jupyter Notebook following that analysis plan.

With this in mind, you can start working with a preliminary dataset that may fit your final project idea, or use the Disney dataset shown in class.

1. **"Project director role"**: review the data in Excel and come up with 3-5 high level questions that could be answered using this dataset. Think of questions that are interesting, realistic, and require digging a few steps deep into the data (i.e., it couldn't be answered easily with one function or

chart in Excel). For example, with Disney data, this could be: "Which were the highest rated movies/directors in each decade?", "Does the release month/season correlate with movie success?", "Does the average rating increase over time for a given director/actor/writer?", "Does having more genres/languages correspond to higher or lower ratings?"

2. **"Project manager role"**: start with one question from the above and describe the question much more precisely in preparation for data analysis, e.g., "How should we define movie success?", "Which date variable is most relevant?", "Should we weight IMDB rating by number of votes?", "Should we give some value to the awards in addition to public scores?" Answering these questions will help you limit your attention to 4-5 variables used for the analysis.
3. **"Data analyst role"**: look at the data again in Excel, while focusing on these core variables defined above, and try to understand what data cleaning will be required. Define the exact steps in writing in sufficient detail. This is known as *analysis plan* or *pseudocode* (when describing an algorithm).
 - I will use *released_at* variable and convert it to datetime type. Based on *released_at*, I will create a variable *decade*.
 - I will create a *combined_rating* variable, using the formula $(\text{metascore}/100 + \text{imdb_rating}/10)/2$. I will treat missing values as...
 - I will create *awards_value* variable by assigning 3 points for each win and 1 point for each nomination. This will require splitting *awards* variable into 2 variables with the number of wins and number of nominations separately.
 - I will split *genre* variable into *genre_1*, *genre_2*, ..., *genre_N* variables. I will also create a variable *genre_count* based on the number of *genres* listed.
 - Relatedly, I will write a custom function that splits any column into multiple columns by comma and also counts the number of columns after splitting, so that I can use this function both for genres, languages, and actors. I will look into *pandas* references to see if a similar function already exists.

Bonus: in general, think of how to make all of your analysis more flexible with user-defined functions, e.g., can you write a function that produces a similar analysis for director/writer/actor, using just *imdb_rating* or *metascore* or the average, aggregated by week/month/year of release?

4. **"Back-and-forth between roles"**: oftentimes, after these first 3 steps, there is a discussion of what is feasible and how much time different analyses would take. If data cleaning to answer one specific business question takes 2 days, but you could answer 5 other similar questions within 2 hours, then you can inform your project manager about these estimates, and ask about the priorities from the project director's perspective. Before giving these estimates, you want to make sure that your process is efficient. Similarly, some questions might simply be impossible to answer (e.g., some key variable is missing for 80% of observations), then you might need to ask for additional data or discuss the methodology on how to treat those missing values. You want to get to this step #4 quite fast, therefore, step #3 usually involves very little actual coding (only high-level inspection with functions like *describe()*, *unique()*, etc.). However, to give accurate estimates you need to know exactly what steps you *would* take (hence, writing a detailed analysis plan) and *how* you would code it.
5. **"Programmer/analyst role"**: after agreeing on the exact questions - go and code. You will realize that when the process is clear, this final step is quite fast. Define a few intermediate checkpoints based on the analysis plan and produce tangible output, which can be helpful for the remaining parts of the analysis.

As you can see, steps 1-4 take most of the space - and they should! In reality, they are the difficult part of the project. Once you define your questions clearly and have a good analysis plan, the coding part should go smoothly. Of course, you always run into technical difficulties once you start coding, but you can resolve those within a couple of hours, whereas, if you do not have a clear plan, if you do not see what the final output should look like, if you forget to note down and discuss the assumptions throughout your analysis - then you can end up spending weeks and months on a project that eventually goes nowhere. Avoiding that is the most difficult skill to learn.

From an educational perspective, learning how to apply specific *pandas* functions (step #5) is important, but it can be easily self-taught following YouTube videos or simply asking AI to write the code. What is lacking in most other data analysis courses, is this deeper understanding of how to think carefully about data, how to ask the right business or research questions, how to align on the end-goals and communicate clearly between different team members, how to plan your analysis in an efficient way, etc. Therefore, I highly recommend practicing these skills throughout our course, in addition to learning the technical details on how to convert your ideas into output with the right coding tools.

Optional: Intro to Streamlit

Try converting some of your code from Jupyter notebooks into a Streamlit app. You can start by displaying some dataset, performing a few data cleaning steps, and displaying the new "cleaner" dataset. Pay attention to relative paths such that the app could read the data when you deploy it on [Streamlit community cloud](#).

A few helpful links about Streamlit:

- Streamlit official playground with basic examples: [link](#)
- Streamlit gallery for inspiration: [link](#)
- Intro to Streamlit YouTube tutorial: [link](#)
- 30 days of Streamlit challenge: [link](#)
- Videos about philosophy of [fast prototyping](#) and [apps for data analysis](#)